DEVIXO SOLUTIONS

AI/ML Internship Program

Task 01

Python Programming, Data Analysis & Visualization

Laptop Price Dataset Analysis

Student Name: Wajiha Waqar

Internship Domain: AI/ML

Submission Date: 24 October 2026

GitHub: github.com/wajihawaqar903-cpu/Devixo-Solutions-Task-01

Table of Contents

1. Introduction
2. Objective
3. Dataset Name
4. Dataset Source
5. Dataset Description
6. Dataset Size
7. Features/Columns Description
8. Methodology
9. Data Loading
10. Data Cleaning
11. Exploratory Data Analysis
12. Statistical Analysis
13. Data Visualizations
14. Code Screenshots (code listings)
15. Output Screenshots
16. Graph Screenshots
17. Major Findings
18. Ten Insights
19. Conclusion
20. GitHub Repository Link

1. Introduction

This report documents a complete data analysis workflow carried out on a laptop price dataset from Kaggle. The work starts from the raw data file and goes through loading and understanding the data, cleaning and preprocessing, exploratory data analysis (EDA), statistical analysis, data visualization, and insight generation. All results, tables, and charts in this report come from my own notebook (Task_01_Data_Analysis.ipynb) run in Google Colab with Python, Pandas, NumPy, Matplotlib and Seaborn.

2. Objective

The objective of Task 01 is to build practical skills in Python-based data analysis. In this project I aimed to:

- load a real-world dataset and understand its rows, columns and data types;
- find and fix data quality problems such as missing values, duplicates, wrong data types and invalid values;
- calculate descriptive statistics (mean, median, mode, standard deviation, variance) and correlations;
- create ten different visualizations and explain what each of them shows;
- write at least ten insights based on the actual data and organize the project on GitHub.

3. Dataset Name

Laptop Price Dataset (laptop listings with specifications, price and customer ratings).

4. Dataset Source

Kaggle (public dataset, downloaded as a CSV file and loaded into Google Colab).

5. Dataset Description

The dataset contains one row per laptop listing. Each listing describes the laptop brand, processor, memory (RAM), storage (SSD and HDD), operating system, graphics card, laptop category, warranty, touchscreen and MS Office availability, its price, and customer feedback (star rating, number of ratings and number of reviews). I chose this dataset because it mixes numerical and categorical variables, is easy to understand, and supports many meaningful questions, for example which brands are most expensive and which hardware is most common.

6. Dataset Size

Stage	Rows	Columns
Original dataset	823	19
After removing 21 duplicate rows (cleaned dataset)	802	19

7. Features/Columns Description

#	Column	Type	Description
1	brand	Categorical	Laptop brand (8 brands, e.g. ASUS, DELL, APPLE)
2	processor_brand	Categorical	Processor maker: Intel, AMD or M1 (Apple)
3	processor_name	Categorical	Processor model, e.g. Core i5, Ryzen 7 (11 types)
4	processor_gnrtn	Categorical	Processor generation, e.g. 10th, 11th, or Not Available
5	ram_gb	Categorical (text with GB)	Memory size: 4, 8, 16 or 32 GB
6	ram_type	Categorical	Memory type, e.g. DDR4, LPDDR4X, DDR5
7	ssd	Categorical (text with GB)	SSD storage size (0 GB means no SSD)
8	hdd	Categorical (text with GB)	HDD storage size (0 GB means no HDD)
9	os	Categorical	Operating system: Windows, Mac or DOS
10	os_bit	Categorical	Operating system type: 32-bit or 64-bit
11	graphic_card_gb	Categorical (text with GB)	Dedicated graphics card memory (0 to 8 GB)
12	weight	Categorical	Laptop category label: Casual, ThinNlight or Gaming
13	warranty	Categorical	Warranty: No warranty, 1 year, 2 years, 3 years
14	touchscreen	Categorical	Touchscreen available: Yes or No
15	msoffice	Categorical	MS Office included: Yes or No
16	price	Numerical (int64)	Laptop price as given in the dataset (target variable of interest)
17	rating	Categorical	Customer star rating, 1 star to 5 stars
18	number_of_ratings	Numerical (int64)	Number of customer ratings
19	number_of_reviews	Numerical (int64)	Number of customer reviews

Column types. Identifying numerical, categorical and date/time columns is important because each type needs different analysis: numerical columns support statistics such as mean and correlation, while categorical columns are analysed with counts, group averages and count plots. The notebook found 3 numerical columns (price, number_of_ratings, number_of_reviews), 16 categorical columns and no date/time columns.

8. Methodology

The project followed the workflow below. Every step was performed in Python inside a Google Colab notebook.

Step	What was done
Dataset selection	Chose the Laptop Price dataset from Kaggle (more than 500 rows, numerical and categorical columns).
Data loading	Imported Pandas, NumPy, Matplotlib and Seaborn and loaded the CSV into a DataFrame.
Data understanding	Viewed first, last and random records, df.info(), shape, column names and column types.
Data cleaning	Checked missing values, removed duplicates, checked and fixed data types, renamed columns, checked invalid values, verified the result.
EDA	Value counts, highest and lowest values, outlier check using the IQR rule and group averages.
Statistical analysis	Mean, median, mode, minimum, maximum, range, standard deviation, variance and the correlation matrix.
Visualization	Created ten charts with titles, axis labels and saved them as PNG files.
Insights and conclusion	Wrote ten insights and the final conclusion from the actual results.
Documentation	Prepared this report and the GitHub repository with README.md.

Tools used: Python 3, Google Colab, Pandas, NumPy, Matplotlib, Seaborn, GitHub.

9. Data Loading

The libraries were imported and the CSV file was uploaded to Colab and read into a DataFrame called *df*. The first five records, the last five records and a random sample of five records were displayed to confirm that the data loaded correctly. The dataset has **823 rows and 19 columns**, and df.info() showed that all 19 columns had 823 non-null values at the start. Three columns are integers (price, number_of_ratings, number_of_reviews) and the other sixteen are text (object) columns.

```
# Load the dataset and view records

import pandas as pd
from google.colab import files

# Upload your CSV file
uploaded = files.upload()

# Load the CSV file into a DataFrame
file_name = next(iter(uploaded))
df = pd.read_csv(file_name)

# Display the first 5 records
print("First 5 Records:")
display(df.head(5))
```

First five records (selected columns):

brand	processor_name	ram_gb	ssd	hdd	os	price	rating
ASUS	Core i3	4 GB	0 GB	1024 GB	Windows	34649	2 stars
Lenovo	Core i3	4 GB	0 GB	1024 GB	Windows	38999	3 stars
Lenovo	Core i3	4 GB	0 GB	1024 GB	Windows	39999	3 stars
ASUS	Core i5	8 GB	512 GB	0 GB	Windows	69990	3 stars
ASUS	Celeron Dual	4 GB	0 GB	512 GB	Windows	26990	3 stars

10. Data Cleaning

Real-world data usually contains missing values, duplicates, wrong data types and invalid values. Each of these was checked as follows.

10.1 Missing Values

Every column was checked for missing values. The result was **0 missing values in all 19 columns (0.0%)**, so no imputation was needed. The notebook still contains the handling code (median for numerical columns and mode for categorical columns), which would run automatically if any missing value existed. No data was removed in this step.

10.2 Duplicate Records

21 duplicate rows were found. Duplicates were removed because the same laptop listing counted twice would distort counts, averages and percentages. After removal the dataset had **802 rows**, and a second check confirmed that 0 duplicate rows remain.

```
# Step 7: Remove Duplicate Rows

# 1. Count duplicate rows
duplicate_count = df.duplicated().sum()
print("Number of duplicate rows:", duplicate_count)

# 2. Remove duplicate rows
df = df.drop_duplicates()

# 3. Reset the index after removing duplicates
df = df.reset_index(drop=True)

# 4. Verify that no duplicates remain
remaining_duplicates = df.duplicated().sum()

print("Duplicate rows after removal:", remaining_duplicates)
print("Total rows after cleaning:", len(df))

if remaining_duplicates == 0:
    print("Success: No duplicate rows remain.")
```

10.3 Data Types

The numerical columns (price, number_of_ratings, number_of_reviews) already had the correct integer type. The 16 text columns were converted to the *category* data type, which is suitable for columns with a limited number of repeated values and uses less memory (memory usage dropped from 122.3 KB to 34.9 KB). No date columns were present, so no date conversion was required.

10.4 Renamed Columns

Column names were standardized to lowercase with underscores so that they are consistent and easy to type. Four columns changed:

Original name	New name
Touchscreen	touchscreen
Price	price
Number of Ratings	number_of_ratings
Number of Reviews	number_of_reviews

```
# Step 9: Rename Columns

# 1. Display original column names
print("Original Column Names:")
print(df.columns.tolist())
```

```
# 2. Standardize column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(r"\s+", "_", regex=True)
)

# 3. Display renamed column names
print("\nRenamed Column Names:")
print(df.columns.tolist())

print("\nColumn names standardized successfully!")
```

10.5 Invalid Values

The notebook checked for negative prices, negative or zero hardware values and invalid categories. **No negative or impossible values were found.** The minimum price is 16,990 and the minimum for ratings and reviews is 0, which is realistic (a new listing can have no ratings yet). All category values (for example brand, os, warranty and rating) were inspected and look valid. The value "Not Available" in processor_gnrtn (224 laptops) is a real label in the data, so it was kept as its own category instead of being deleted.

10.6 Verification of the Cleaned Dataset

Check	Result
Rows	802
Columns	19
Missing values	0
Duplicate rows	0
Data types	3 numerical (int64), 16 category

11. Exploratory Data Analysis

What does the dataset contain? 802 unique laptop listings described by 19 features. **Which variables are important?** price is the main variable of interest, supported by brand, processor, RAM, storage and graphics. The most frequent categories are listed below.

Variable	Most frequent value	Count (of 802)	Share
brand	ASUS	243	30.3%
processor_brand	Intel	594	74.1%
processor_name	Core i5	284	35.4%
processor_gnrtn	11th	328	40.9%
ram_gb	8 GB	404	50.4%
ram_type	DDR4	690	86.0%
ssd	512 GB	389	48.5%
hdd	0 GB (no HDD)	602	75.1%
os	Windows	763	95.1%
graphic_card_gb	0 GB (no dedicated card)	557	69.5%
weight (category)	Casual	509	63.5%
warranty	1 year	498	62.1%
touchscreen	No	706	88.0%
msoffice	No	522	65.1%
rating	4 stars	437	54.5%

Highest and lowest values.

Column	Minimum	Maximum
price	16,990	441,990
number_of_ratings	0	15,279
number_of_reviews	0	1,947

Outliers (IQR rule). For price, the IQR method gives an upper limit of 154,827.5, and **49 laptops** are priced above it. For number_of_ratings and number_of_reviews, **128 laptops** each are outliers (upper limits 350.6 and 45). These are mostly premium laptops and a few very popular models, not data errors, so they were kept.

Group averages: average price by brand.

Brand	Average price	Laptops
APPLE	157,081.30	23
MSI	98,713.02	50
ASUS	79,426.02	243
HP	73,640.27	136
Lenovo	72,920.21	144
acer	72,420.04	51
DELL	60,788.95	149
Avita	56,346.50	6

12. Statistical Analysis

12.1 Descriptive Statistics

Statistic	price	number_of_ratings	number_of_reviews
Mean	76,625.54	299.84	36.09
Median	63,990	17	2
Mode	64,990	0	0
Minimum	16,990	0	0
Maximum	441,990	15,279	1,947
Range	425,000	15,279	1,947
Standard deviation	45,232.98	1,001.78	118.31
Variance	about 2.05 billion (2.046e9)	1,003,572	13,998.1

```
# Step 12: Descriptive Statistics

import pandas as pd
import numpy as np

# Select numerical columns
numeric_df = df.select_dtypes(include=["number"])

# Calculate descriptive statistics
statistics_table = pd.DataFrame({
    "Mean": numeric_df.mean(),
    "Median": numeric_df.median(),
    "Mode": numeric_df.mode().iloc[0] if not numeric_df.empty else pd.Series(dtype=float),
    "Minimum": numeric_df.min(),
    "Maximum": numeric_df.max(),
    "Range": numeric_df.max() - numeric_df.min(),
    "Standard Deviation": numeric_df.std(),
    "Variance": numeric_df.var()
})

# Display results in one table
print("Descriptive Statistics for Numerical Columns:")
display(statistics_table.round(2))
```

12.2 Mean

The mean is calculated as **Mean = (sum of all values) / n**, where n is the number of observations. The average laptop price is **76,625.54**. This shows the typical price level of the market in this dataset, but it is pulled upward by a few very expensive laptops.

12.3 Median

The median price is **63,990**, about 12,636 lower than the mean (the mean is roughly 20% above the median). The median is more useful when extreme values are present because it is not affected by them. For number_of_ratings the difference is even bigger: the mean is 299.84 but the median is only 17, which means most laptops have very few ratings while a small number have thousands.

12.4 Mode

The most common price is 64,990. For the categorical columns the modes are: brand ASUS, processor Intel Core i5, 11th generation, 8 GB RAM, DDR4, 512 GB SSD, no HDD, Windows 64-bit, no dedicated graphics, 1 year warranty, no touchscreen, no MS Office and a 4 star rating.

12.5 Standard Deviation

The standard deviation of price is **45,232.98**, which is large compared with the mean of 76,625.54, so prices vary a lot between laptops. In relative terms number_of_ratings (standard deviation 1,001.78 against a mean of 299.84) varies even more, because a few listings have thousands of ratings.

12.6 Correlation Analysis

	price	number_of_ratings	number_of_reviews
price	1.00	-0.15	-0.16
number_of_ratings	-0.15	1.00	0.99
number_of_reviews	-0.16	0.99	1.00

The correlation coefficient lies between -1 and +1. The strongest correlation is between **number_of_ratings and number_of_reviews (0.991)**, a very strong positive relationship: laptops that get many ratings also get many reviews. Price has a weak negative correlation with number_of_reviews (-0.157) and number_of_ratings (-0.153), meaning cheaper laptops tend to collect slightly more customer feedback, but the relationship is too weak to be a strong rule.

13. Data Visualizations

Ten visualizations were created. Each chart has a title, axis labels and a readable size, and was saved as a PNG file (visualization_01.png to visualization_10.png) in the images folder of the repository.

13.1 Histogram: Distribution of Laptop Prices

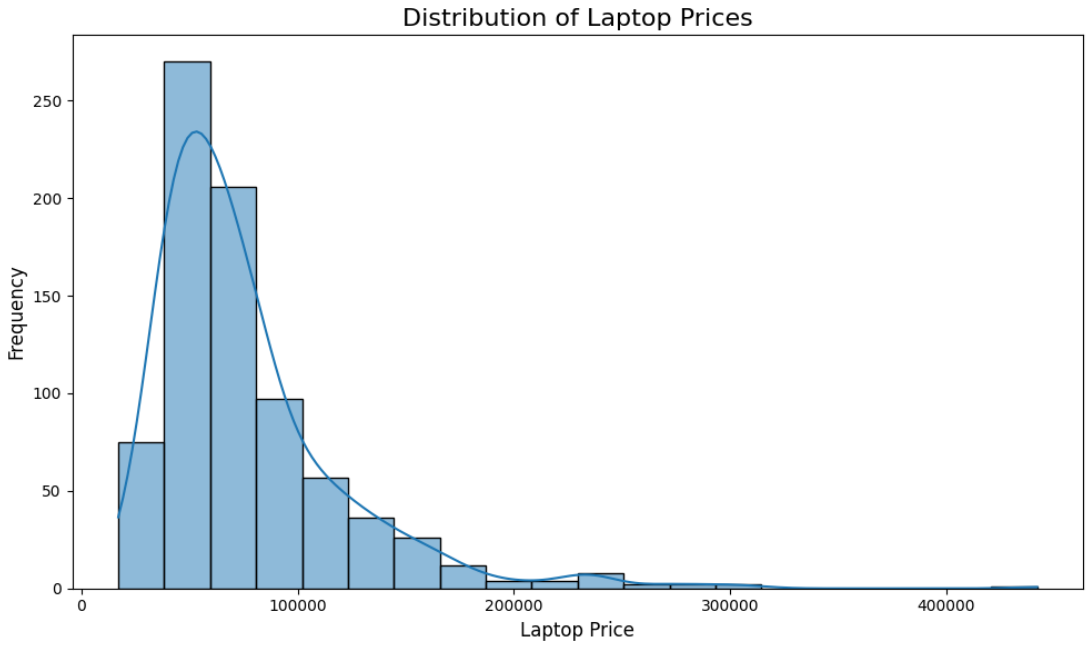


Figure: Histogram: Distribution of Laptop Prices

Interpretation: The histogram shows how laptop prices are spread. The distribution is right-skewed: most laptops are priced between about 40,000 and 100,000, with the tallest bar near 40,000 to 60,000, and the number of laptops falls steadily as price increases up to 441,990. This matches the mean (76,625) being higher than the median (63,990). It matters because a typical buyer will find most laptops in the lower price range, while premium models are rare.

13.2 Bar Chart: Average Laptop Price by Brand

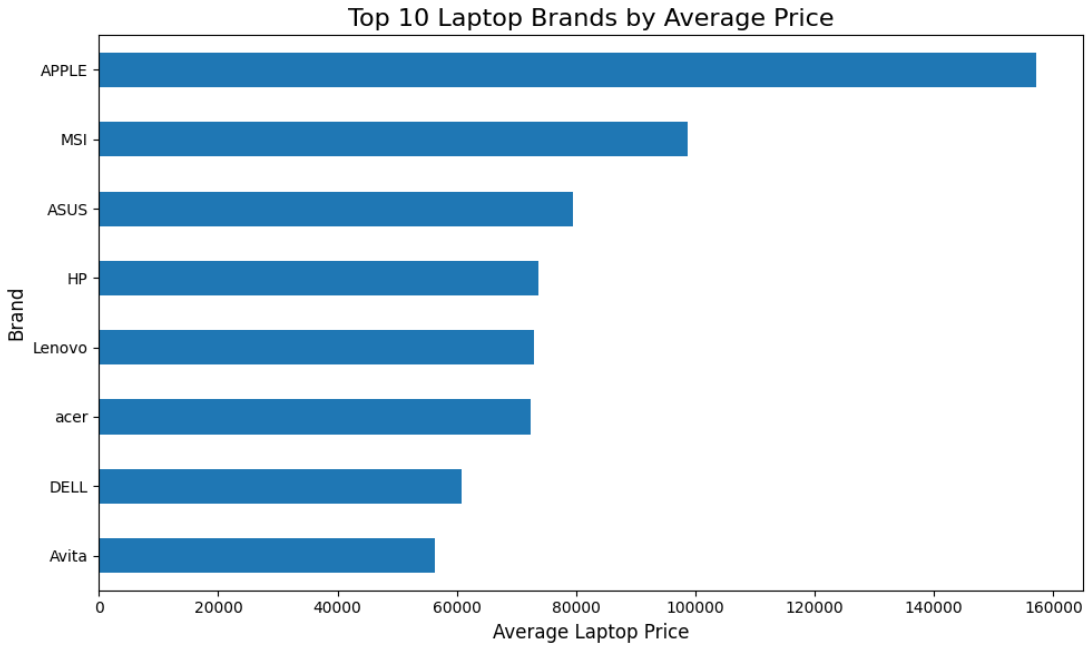


Figure: Bar Chart: Average Laptop Price by Brand

Interpretation: APPLE has the highest average price (157,081), roughly 2.8 times that of Avita (56,346), the lowest. MSI (98,713) comes second, followed by ASUS, HP, Lenovo and acer in the 72,000 to 79,000 band, with DELL at 60,789. This shows that brand positioning strongly affects price: Apple and MSI sell premium or gaming devices while DELL and Avita sit at the budget end.

13.3 Pie Chart: Distribution of Laptop Brands

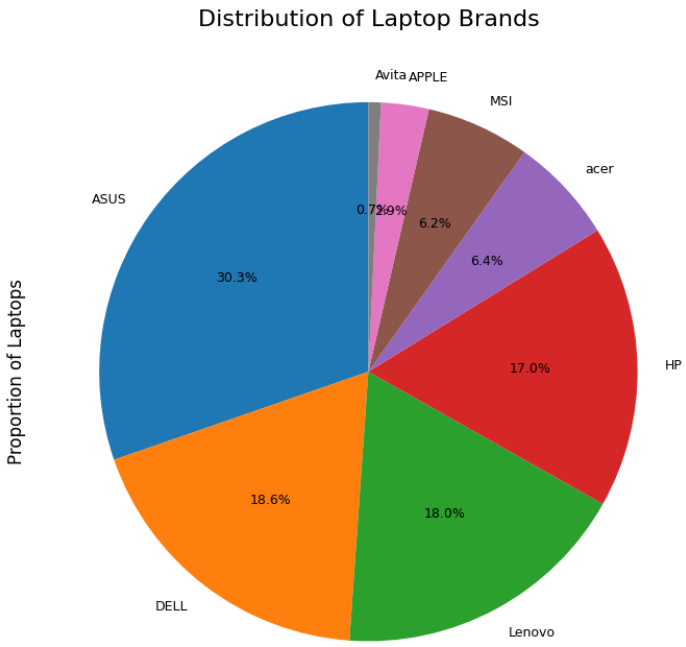


Figure: Pie Chart: Distribution of Laptop Brands

Interpretation: ASUS holds the largest share of listings (30.3%), followed by DELL (18.6%), Lenovo (18.0%) and HP (17.0%). These four brands cover about 84% of the dataset, while acer (6.4%), MSI (6.2%), APPLE (2.9%) and Avita (0.7%) are small. The market in this dataset is therefore dominated by a few well-known brands.

13.4 Scatter Plot: RAM vs Laptop Price

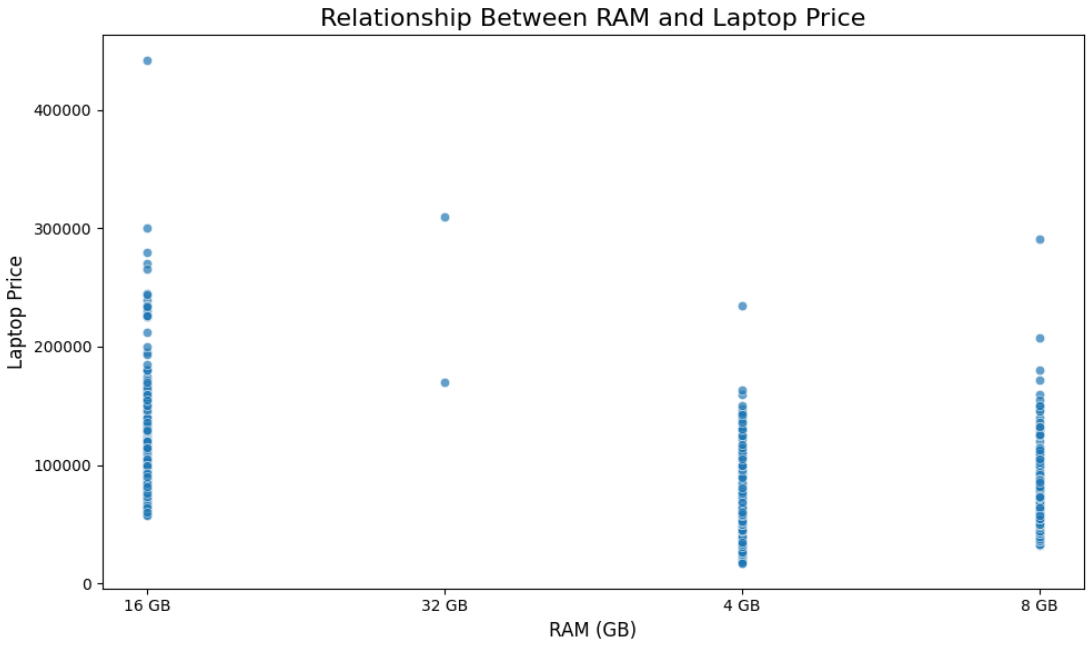


Figure: Scatter Plot: RAM vs Laptop Price

Interpretation: Each dot is one laptop. Because RAM is stored as text, the categories appear in text order on the x-axis (16 GB, 32 GB, 4 GB, 8 GB). Laptops with 4 GB RAM are concentrated at lower prices (mostly below 150,000), 8 GB laptops cover a wider range, and 16 GB laptops start from about 58,000 and reach the highest price of 441,990. The two 32 GB laptops are expensive (about 170,000 and 310,000). Overall, more RAM goes with higher prices.

13.5 Line Chart: Average Laptop Price by RAM Size

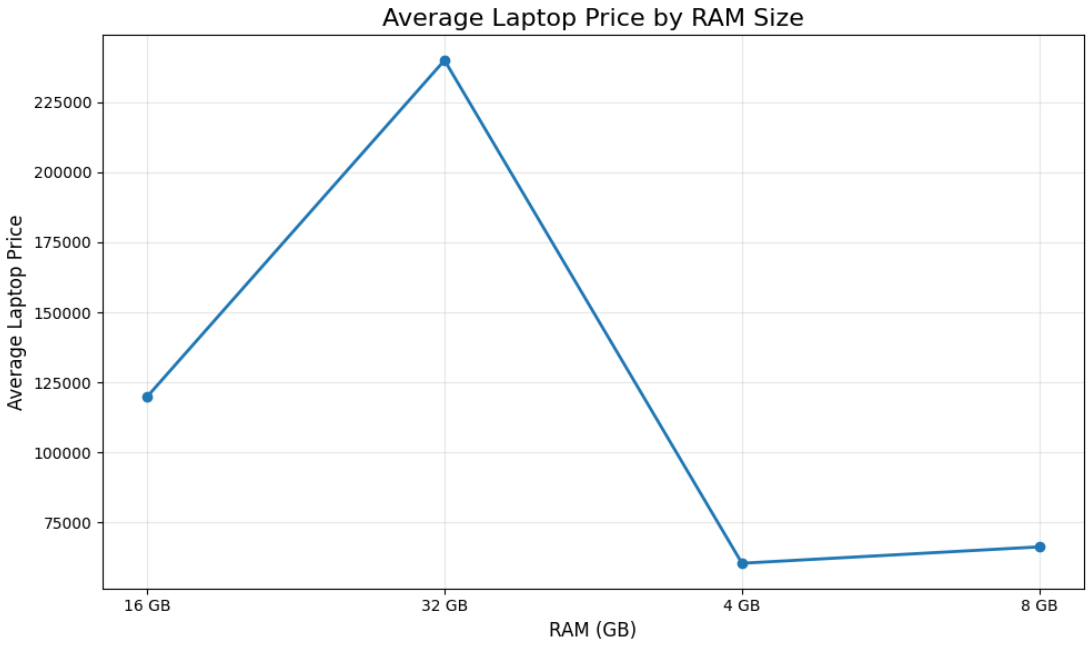


Figure: Line Chart: Average Laptop Price by RAM Size

Interpretation: The average price is about 60,000 for 4 GB, about 66,000 for 8 GB, about 120,000 for 16 GB and about 240,000 for 32 GB, so the average price rises as RAM increases. The chart lists the RAM sizes in text order (16, 32, 4, 8 GB), which is why the line moves up and down; read in numerical order the trend is steadily upward. The 32 GB value is based on only 2 laptops, so it should be treated with care.

13.6 Box Plot: Laptop Price Outliers

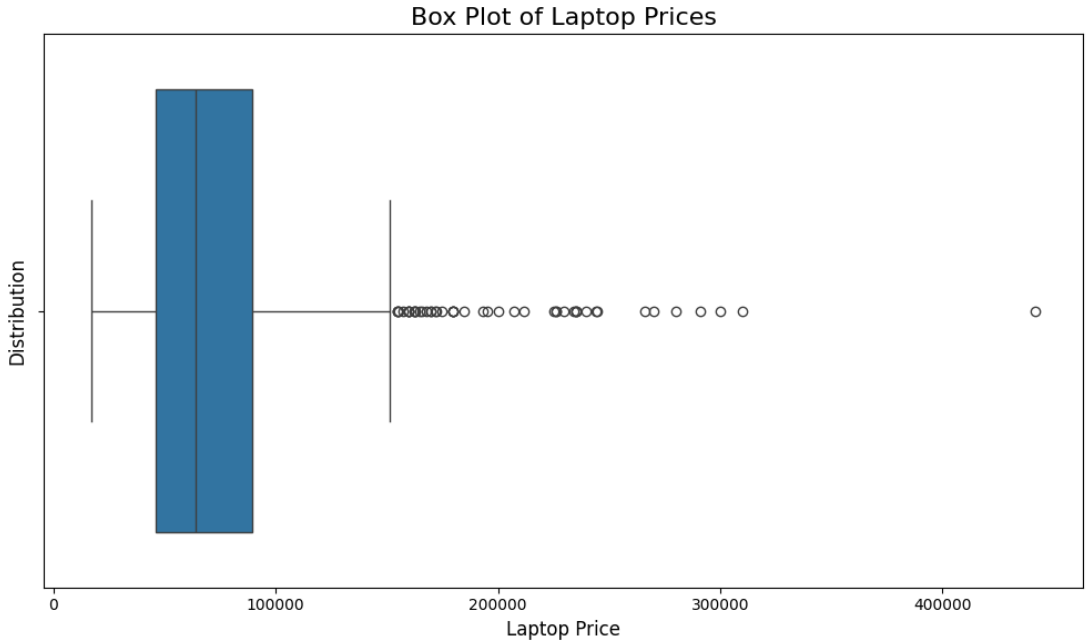


Figure: Box Plot: Laptop Price Outliers

Interpretation: The box covers the middle 50% of prices (about 45,990 to 89,525) with the median line at 63,990. The whisker on the right ends at about 152,000 and many points beyond it are outliers: 49 laptops above 154,827, with the most expensive at 441,990. The plot confirms that the price distribution has a long upper tail.

13.7 Correlation Heatmap of Numerical Features

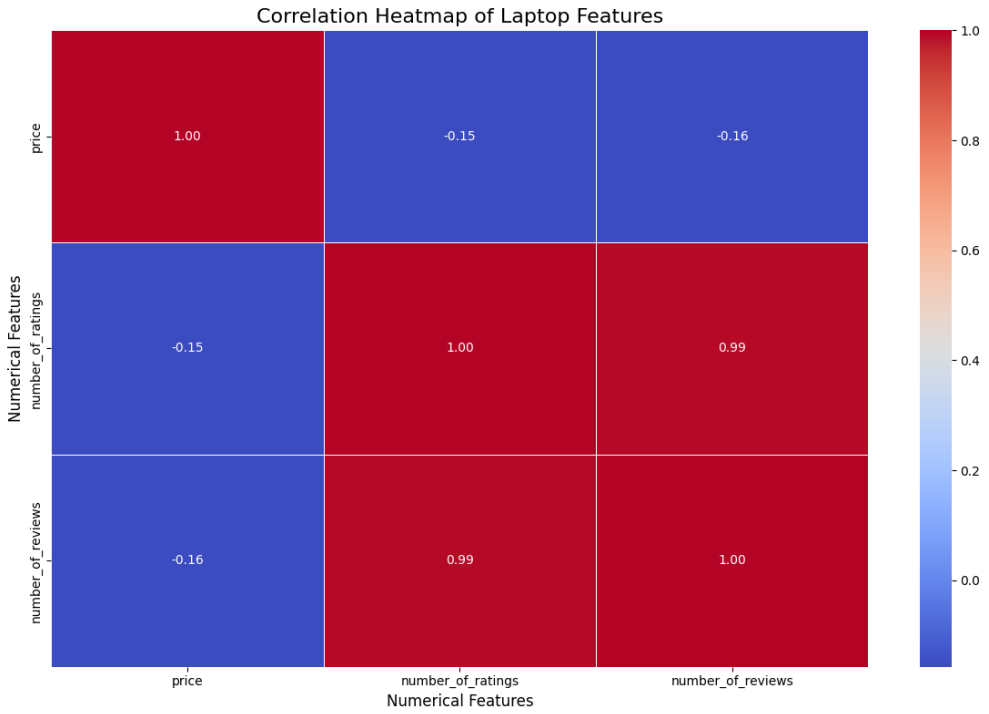


Figure: Correlation Heatmap of Numerical Features

Interpretation: The heatmap shows number_of_ratings and number_of_reviews are almost perfectly correlated (0.99), while price has weak negative correlations with both (-0.15 and -0.16). This means popularity measures move together but price does not strongly explain how many ratings a laptop receives.

13.8 Count Plot: Number of Laptops by Brand

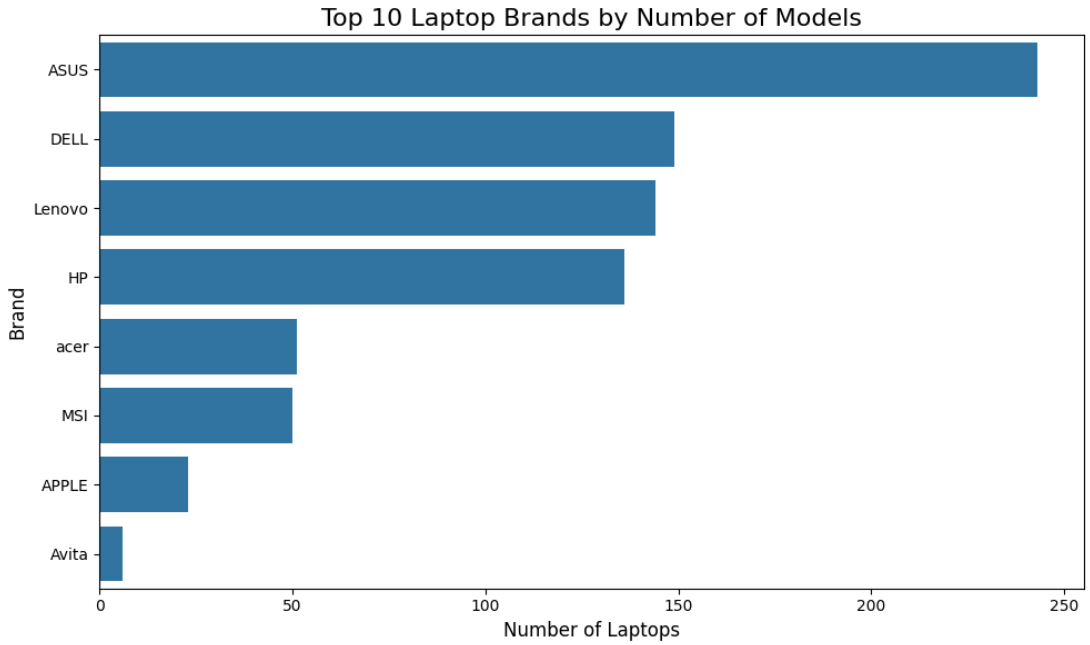


Figure: Count Plot: Number of Laptops by Brand

Interpretation: ASUS has the most laptop listings (243), followed by DELL (149), Lenovo (144) and HP (136). acer (51) and MSI (50) are mid-sized, while APPLE (23) and Avita (6) have the fewest. The chart gives the exact counts behind the pie chart and shows that product variety is concentrated in four brands.

13.9 Pair Plot of Laptop Features (price, RAM, SSD, HDD, graphics)

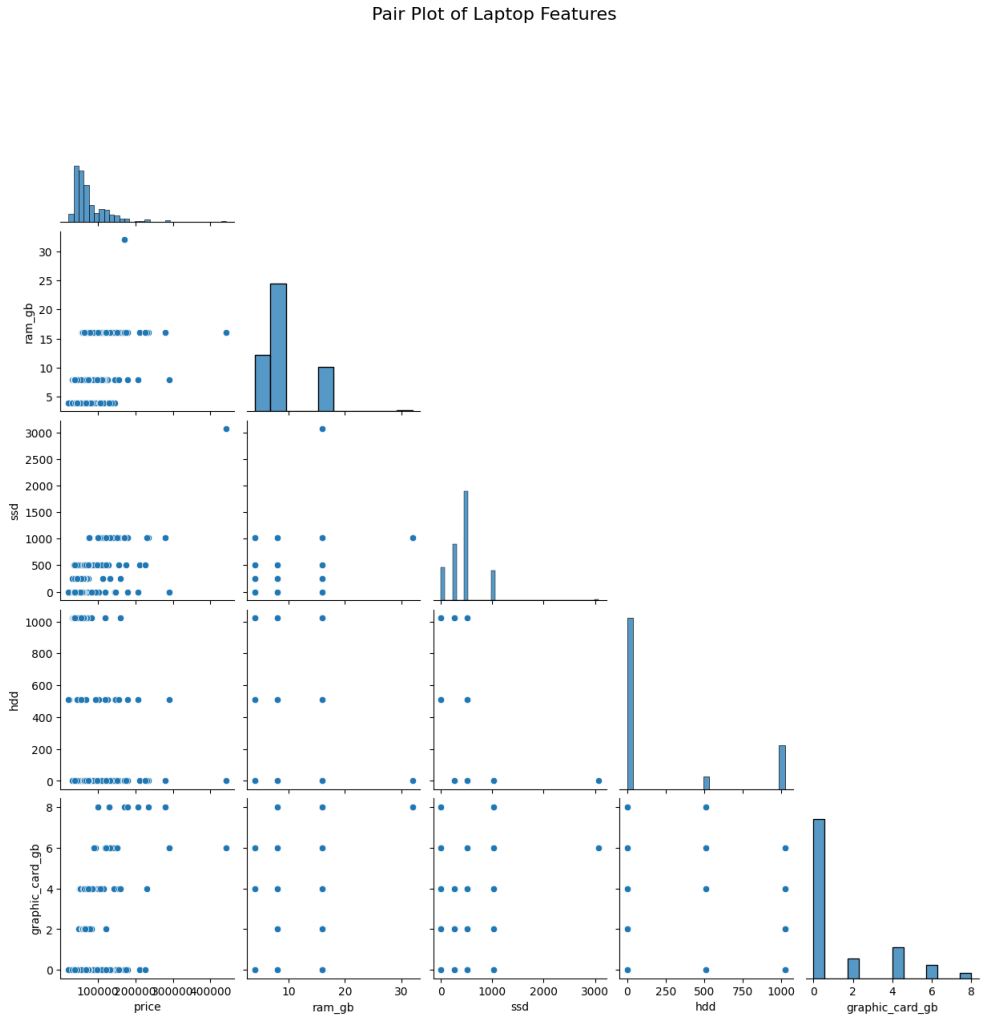


Figure: Pair Plot of Laptop Features (price, RAM, SSD, HDD, graphics)

Interpretation: The pair plot compares price, RAM, SSD size, HDD size and graphics card memory (a sample of 300 laptops). The price histogram is right-skewed, RAM is mostly 4, 8 or 16 GB, and HDD is split between no HDD and 1024 GB. Higher price points appear with larger RAM, bigger SSDs and dedicated graphics, while laptops with 1024 GB HDD sit mostly at lower prices. The most expensive laptop in the sample has a very large SSD (about 3,000 GB).

13.10 Violin Plot: Laptop Price Distribution by Brand

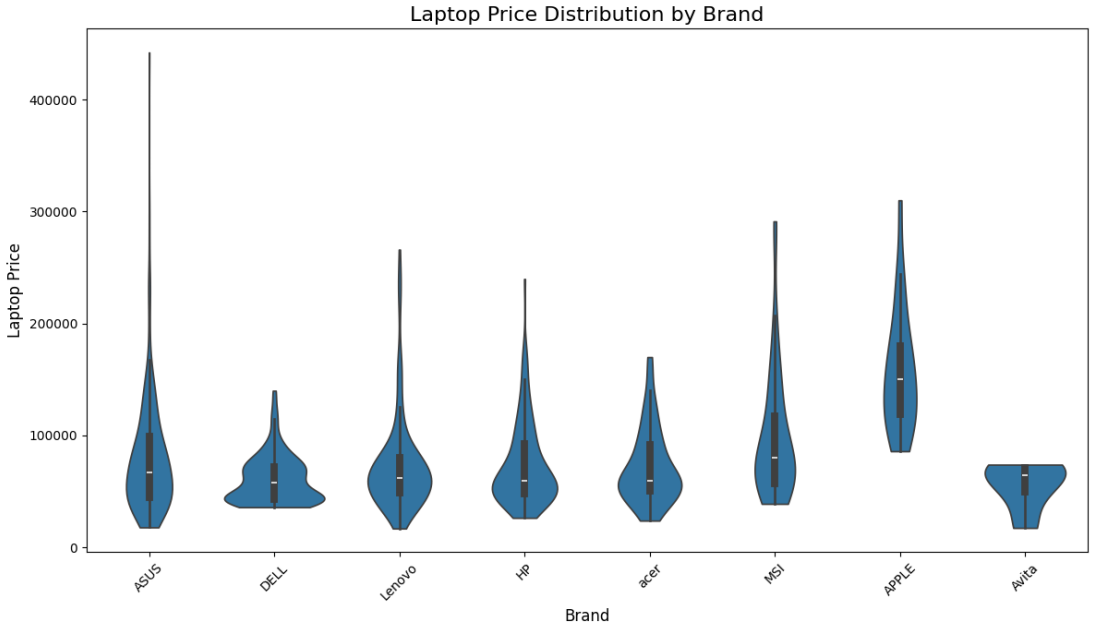


Figure: Violin Plot: Laptop Price Distribution by Brand

Interpretation: The violin plot shows the full price distribution of each brand. APPLE prices are concentrated high (median about 150,000) with a minimum of about 85,000, so even the cheapest Apple laptop costs more than the typical laptop of other brands. ASUS has the widest spread with a very long tail up to 441,990. DELL is tightly grouped below about 140,000, and Avita is narrow and low. This shows that brands differ not only in average price but also in how much choice they offer across price levels.

14. Code Screenshots (code listings)

The main code sections used in the notebook are listed below. Screenshots of the same cells from Google Colab can be inserted next to them if required.

Missing value check and handling

```
# Step 6: Check and Handle Missing Values

import pandas as pd
import numpy as np

# 1. Count missing values per column
missing_count = df.isnull().sum()

# 2. Calculate missing value percentage per column
missing_percentage = (df.isnull().sum() / len(df)) * 100

# 3. Display missing values report
missing_report = pd.DataFrame({
    "Missing Values": missing_count,
    "Missing Percentage (%)": missing_percentage.round(2)
})

print("Missing Values Report:")
display(missing_report)

# 4. Handle missing values
for column in df.columns:

    if df[column].isnull().any():

        # Fill numerical columns with median
        if pd.api.types.is_numeric_dtype(df[column]):
            df[column] = df[column].fillna(df[column].median())

        # Fill categorical columns with mode
        else:
            mode_values = df[column].mode(dropna=True)

            if not mode_values.empty:
                df[column] = df[column].fillna(mode_values.iloc[0])
            else:
                print(f"Column '{column}' contains only missing values.")

# 5. Display missing values after handling
print("\nMissing Values After Cleaning:")
print(df.isnull().sum())

print("\nTotal Missing Values:", df.isnull().sum().sum())
```

Fixing data types

```
# Step 8: Fix Data Types

import pandas as pd
import numpy as np

# 1. Display data types before conversion
print("Data Types Before Conversion:")
print(df.dtypes)

# 2. Convert numeric-looking text columns into numbers
for column in df.select_dtypes(include=["object", "string"]).columns:
    cleaned = df[column].astype("string").str.strip()

    # Remove common formatting from numeric values, such as commas
    numeric_text = cleaned.str.replace(",", "", regex=False)
    numeric_text = numeric_text.str.replace(r"[$£€]", "", regex=True)

    converted = pd.to_numeric(numeric_text, errors="coerce")

    # Convert only if all non-missing original values are numeric
    if cleaned.notna().sum() > 0 and (
        converted.notna().sum() == cleaned.notna().sum()
    ):
```

```
        df[column] = converted

# 3. Detect columns whose names suggest they contain dates or times
for column in df.columns:
    if any(word in column.lower() for word in ["date", "datetime", "timestamp"]):
        if df[column].dtype == "object" or pd.api.types.is_string_dtype(df[column]):
            converted_dates = pd.to_datetime(df[column], errors="coerce")

            # Convert only if all non-missing values are valid dates
            original_non_null = df[column].notna().sum()
            if original_non_null > 0 and converted_dates.notna().sum() == original_non_null:
                df[column] = converted_dates

# 4. Convert remaining text columns to categorical data
for column in df.select_dtypes(include=["object", "string"]).columns:
    df[column] = df[column].astype("category")

# 5. Display data types after conversion
print("\nData Types After Conversion:")
print(df.dtypes)

# 6. Display final column types
print("\nFinal Data Type Summary:")
print(df.dtypes.value_counts())

print("\nData types checked and conversion completed.")
```

Outlier check and group averages (EDA)

```
# Step 15: Exploratory Data Analysis (EDA)

import pandas as pd
import numpy as np

# 1. Value counts for categorical columns
print("1. VALUE COUNTS FOR CATEGORICAL COLUMNS")

categorical_columns = df.select_dtypes(
    include=["object", "category", "string"]
).columns

for column in categorical_columns:
    print(f"\nValue counts for {column}:")
    display(df[column].value_counts(dropna=False).head(10))

# 2. Highest and lowest values in numerical columns
print("\n2. HIGHEST AND LOWEST VALUES")

numeric_df = df.select_dtypes(include="number")

for column in numeric_df.columns:
    values = numeric_df[column].dropna()

    if not values.empty:
        print(f"\nColumn: {column}")
        print("Minimum value:", values.min())
        print("Maximum value:", values.max())

# 3. Outlier detection using the IQR method
print("\n3. OUTLIER CHECK USING IQR")

for column in numeric_df.columns:
    values = numeric_df[column].dropna()

    if len(values) < 4:
        print(f"{column}: Not enough values for a useful IQR check.")
        continue

    Q1 = values.quantile(0.25)
    Q3 = values.quantile(0.75)
    IQR = Q3 - Q1

    lower_limit = Q1 - 1.5 * IQR
    upper_limit = Q3 + 1.5 * IQR

    outliers = values[
        (values < lower_limit) | (values > upper_limit)
    ]
```

```
    print(f"\nColumn: {column}")
    print("Lower limit:", round(lower_limit, 2))
    print("Upper limit:", round(upper_limit, 2))
    print("Number of outliers:", len(outliers))

# 4. Average numerical values by categorical groups
print("\n4. GROUP AVERAGES")

if len(categorical_columns) > 0 and len(numeric_df.columns) > 0:
    # Use the first categorical column and the first numerical column
    category_column = categorical_columns[0]
    numeric_column = numeric_df.columns[0]

    print(f"\nAverage {numeric_column} by {category_column}:")

    group_averages = df.groupby(
        category_column,
        observed=True
    )[numeric_column].mean().sort_values(ascending=False)

    display(group_averages.head(10).round(2))

else:
    print("Categorical or numerical columns are not available.")

print("\nEDA analysis completed successfully!")
```

15. Output Screenshots

Key outputs produced by the notebook are reproduced below.

Dataset information (original)

```
DATASET SHAPE
Number of rows: 823
Number of columns: 19

TOTAL MISSING VALUES
0

Number of duplicate rows: 21
Duplicate rows after removal: 0
Total rows after cleaning: 802
```

Cleaning verification summary

```
7. CLEANING VERIFICATION SUMMARY
Rows: 802
Columns: 19
Missing values: 0
Duplicate rows: 0

Dataset verification completed!
```

Descriptive statistics

```
                       Mean   Median   Mode  Minimum  Maximum   Range
price              76625.54  63990.0  64990    16990   441990  425000
number_of_ratings    299.84     17.0      0        0    15279   15279
number_of_reviews     36.09      2.0      0        0     1947    1947

                   Standard Deviation      Variance
price                       45232.98  2.046023e+09
number_of_ratings            1001.78  1.003572e+06
number_of_reviews             118.31  1.399810e+04
```

Correlation analysis

```
Strongest Positive Correlations:
number_of_ratings  number_of_reviews    0.991

Strongest Negative Correlations:
price  number_of_reviews   -0.157
       number_of_ratings   -0.153
```

Outlier check (IQR)

```
price               lower limit -19312.5, upper limit 154827.5, outliers: 49
number_of_ratings   lower limit -210.38,  upper limit 350.62,   outliers: 128
number_of_reviews   lower limit -27.0,    upper limit 45.0,     outliers: 128
```

16. Graph Screenshots

All ten graphs (histogram, bar chart, pie chart, scatter plot, line chart, box plot, correlation heatmap, count plot, pair plot and violin plot) are shown with their interpretations in Section 13 and are also saved in the images folder of the GitHub repository.

17. Major Findings

- The dataset has 802 unique laptops after removing 21 duplicates, and it has no missing values.
- Prices are right-skewed: the mean (76,626) is higher than the median (63,990), and 49 laptops are price outliers.
- APPLE is the most expensive brand on average (157,081) and Avita the cheapest (56,347); ASUS has the most listings (243).
- Larger RAM goes with higher prices: roughly 60,000 on average for 4 GB, 66,000 for 8 GB, 120,000 for 16 GB.
- Number of ratings and number of reviews are almost perfectly correlated (0.99), and price is only weakly related to either (-0.15 and -0.16).

18. Ten Insights

The following twelve insights (more than the required ten) are based on the actual results of my analysis.

Insight 1: Dataset size and quality. After removing 21 duplicates the dataset has 802 laptops and 19 columns, with 0 missing values. *Implication:* The data is reliable enough for analysis without imputation.

Insight 2: ASUS dominates the listings. ASUS has 243 laptops (30.3%), more than DELL (149), Lenovo (144) and HP (136). *Implication:* ASUS offers the widest choice of models, so buyers have the most options with this brand.

Insight 3: Four brands cover most of the market. ASUS, DELL, Lenovo and HP together account for 672 of 802 laptops (about 84%). *Implication:* The market is concentrated; small brands like Avita (6 laptops, 0.7%) have very little presence.

Insight 4: Apple is the premium brand. APPLE has the highest average price (157,081.30), about 2.8 times Avita (56,346.50) and 2.6 times DELL (60,788.95). *Implication:* Brand strongly influences price, and Apple buyers pay a clear premium.

Insight 5: Prices are right-skewed. The mean price is 76,625.54 while the median is 63,990, about 20% lower, and the price range is 16,990 to 441,990. *Implication:* The median describes a typical laptop better than the mean for this data.

Insight 6: Many expensive outliers. 49 laptops are priced above the IQR upper limit of 154,827.5, with the maximum at 441,990. *Implication:* A small group of high-end laptops pulls the average up and should be treated separately in pricing studies.

Insight 7: Intel is the leading processor brand. Intel appears in 594 laptops (74.1%), AMD in 189 (23.6%) and Apple M1 in 19 (2.4%); Core i5 is the most common processor (284 laptops). *Implication:* Most buyers and sellers rely on Intel Core i5 as the standard mid-range choice.

Insight 8: 8 GB RAM is the standard. 8 GB RAM is found in 404 laptops (50.4%), 4 GB in 223 (27.8%) and 16 GB in 173 (21.6%); 32 GB appears in only 2 laptops. *Implication:* Half of the market uses 8 GB, so it is the practical baseline for a new laptop.

Insight 9: Most laptops have no dedicated graphics. 557 laptops (69.5%) have 0 GB graphics memory and 602 (75.1%) have no HDD; SSD storage of 512 GB is the most common (389 laptops, 48.5%). *Implication:* Modern laptops favour SSD storage and integrated graphics; dedicated graphics is a feature of a smaller, usually pricier group.

Insight 10: Windows is almost universal. Windows runs on 763 laptops (95.1%), Mac on 23 and DOS on 16; 693 laptops (86.4%) use a 64-bit system. *Implication:* Software compatibility is mainly a Windows question, and a few budget laptops ship with DOS and need an OS purchase.

Insight 11: Ratings and reviews move together. The correlation between number_of_ratings and number_of_reviews is 0.991, while price has only weak correlations with them (-0.153 and -0.157). *Implication:* Popularity measures are almost the same information, and price does not explain customer engagement.

Insight 12: Customer ratings are mostly average to good, and engagement is uneven. 4 stars is the most common rating (437 laptops, 54.5%) and only 16 laptops (2.0%) have 5 stars; the median number of ratings is 17 while the mean is 299.8 and the maximum 15,279. *Implication:* Most laptops have little feedback, so a few popular models receive most of the attention.

19. Conclusion

In this project I selected the Laptop Price dataset from Kaggle, which describes 823 laptop listings with 19 features, and performed a complete data analysis workflow. During cleaning I checked missing values (none found), removed 21 duplicate rows to get 802 unique laptops, converted text columns to the category type, standardized four column names and confirmed that there were no invalid values. In the exploratory and statistical analysis I calculated the mean, median, mode, standard deviation, variance and correlations, checked outliers with the IQR rule, and compared average prices by brand. Ten visualizations (histogram, bar chart, pie chart, scatter plot, line chart, box plot, correlation heatmap, count plot, pair plot and violin plot) were created and interpreted.

The most important findings are that laptop prices are right-skewed with a median of 63,990 and many high-priced outliers, that Apple laptops are by far the most expensive on average while ASUS offers the most models, that larger RAM goes with higher prices, and that ratings and reviews are almost perfectly correlated (0.99). From this task I learned how to explain every cleaning decision, how to read statistics such as mean versus median, and how to turn charts into clear written findings. A limitation is that memory and storage columns are stored as text with GB units, so the correlation analysis only covered three numerical columns; converting them to numbers would allow a deeper analysis of price and hardware in future work. This type of analysis can help buyers compare laptops and help sellers understand pricing in the market.

20. GitHub Repository Link

https://github.com/wajihawaqar903-cpu/Devixo-Solutions-Task-01

Repository structure:

```
Devixo-Solutions-Task-01/
  dataset/    (laptop dataset CSV)
  notebook/   Task_01_Data_Analysis.ipynb
  images/     visualization_01.png ... visualization_10.png
  report/     Task_01_Report.pdf
  README.md
```